

KONGU ENGINEERING COLLEGE
(AUTONOMOUS)

PERUNDURAI, ERODE – 638 060

EXPERIMENT - 5

PROJECT TITLE

**WATER LEVEL DETECTION AND FLOOD ALERT SYSTEM
USING ULTRASONIC SENSOR**

SUBMITTED BY

ABISHEK R S - 24MER002

GOWRISANKAR K - 24MER010

NAVEENRAJ K - 24MER035

PRAVEEN N M - 24MER043

WATER LEVEL DETECTION AND FLOOD ALERT SYSTEM USING ULTRASONIC SENSOR

1. Project review :

The Water Level Detection and Flood Alert System Using Ultrasonic Sensor is a compact prototype designed to continuously monitor the water level in a river, drainage system, tank, or flood-prone area. The system uses an ESP8266 NodeMCU as the central controller and an ultrasonic sensor to measure the distance between the sensor and the water surface.

The ultrasonic sensor continuously measures the distance to the water surface and sends the measured value to the ESP8266 NodeMCU. The controller processes the distance value and compares it with predefined water-level limits. When the water level rises above the selected danger level, a buzzer provides an audible warning, and an LED or relay module can be activated for flood alert indication.

The proposed system provides a simple, low-cost, and automatic method for water-level monitoring and flood warning. It can also be further developed using Wi-Fi connectivity, cloud monitoring, mobile notifications, and data logging for a complete IoT-based flood monitoring system.

2. Problem Statement :

Flooding and excessive water levels can cause damage to property, infrastructure, agriculture, and human life. In many locations, continuous monitoring of water levels may not be available, making it difficult to provide an immediate warning when the water level rises to a dangerous condition.

Manual monitoring of water levels is inconvenient and may not provide a timely warning during sudden rainfall or flooding. Therefore, there is a need for a simple and low-cost system that can automatically detect changes in water level and provide an immediate flood alert.

The proposed system addresses these problems by integrating an ultrasonic sensor, ESP8266 NodeMCU, buzzer, LED, and relay module. The ultrasonic sensor measures the distance between the sensor and the water surface, while the ESP8266 processes the measured data. When the water level reaches a predefined danger level, the buzzer and LED provide an alert, and the relay can be used for suitable warning or control applications.

3. Proposed Solution :

The proposed solution consists of an ESP8266-based Water Level Detection and Flood Alert System. The ultrasonic sensor is installed above the water surface and continuously measures the distance between the sensor and the water.

As the water level rises, the distance between the ultrasonic sensor and the water surface decreases. The ESP8266 processes the measured distance and compares it with predefined safety limits. If the measured distance indicates that the water has reached a dangerous level, the ESP8266 activates the buzzer and LED to provide a warning. A relay module can also be activated for additional alert or control purposes.

Proposed Operation:

1. Start the system and initialize the ESP8266.
2. Initialize the ultrasonic sensor, buzzer, LED, and relay.
3. The ultrasonic sensor sends an ultrasonic pulse.
4. The pulse is reflected from the water surface.
5. The sensor measures the echo time.
6. ESP8266 calculates the distance between the sensor and water surface.
7. The water level is determined from the measured distance.
8. The measured value is compared with predefined safety limits.
9. If the water reaches the warning or danger level, the buzzer and LED are activated.
10. The relay can be activated for suitable flood warning or control.
11. If the water level is normal, the warning devices remain OFF.
12. The system continuously repeats the monitoring process.

4. System Architecture :

The Water Level Detection and Flood Alert System consists of five main sections:

Input Section:

The input section consists of:

- **Ultrasonic Sensor** – Measures the distance between the sensor and the water surface.
- The ultrasonic sensor sends the distance information to the ESP8266 NodeMCU.

Processing Section:

The ESP8266 NodeMCU acts as the central controller.

- Sends a trigger signal to the ultrasonic sensor.
- Receives the echo signal.
- Calculates the distance to the water surface.
- Determines the water level.
- Compares the water level with predefined limits.
- Controls the buzzer, LED, and relay according to the programmed condition.

Actuation Section:

The Relay Module is used for additional flood warning or control.

- The relay receives a control signal from the ESP8266.
- It can control a suitable alarm, warning light, pump, or other connected device.

Indication Section:

The Buzzer and LED provide warning indications.

- Buzzer OFF and LED OFF → Normal water level.
- Buzzer ON and LED ON → High water level / Flood alert.

Connectivity Section:

The ESP8266 has built-in Wi-Fi capability.

The system can be further developed for:

- IoT-based flood monitoring.
- Cloud data storage.
- Mobile notifications.
- Remote water-level monitoring.
- Automatic emergency alerts.

5. CIRCUIT TABLE :

Important Pin Mapping:

ESP8266 NodeMCU Pin Component

GPIO 5	D1 Ultrasonic Sensor → TRIG
GPIO 4	D2 Ultrasonic Sensor → ECHO
GPIO 14	D5 Relay Module → IN
GPIO 12	D6 Buzzer → Positive Terminal
GPIO 13	D7 LED → Anode through resistor
5V / VIN	– Ultrasonic Sensor → VCC
GND	– Common Ground

6. Circuit Design :

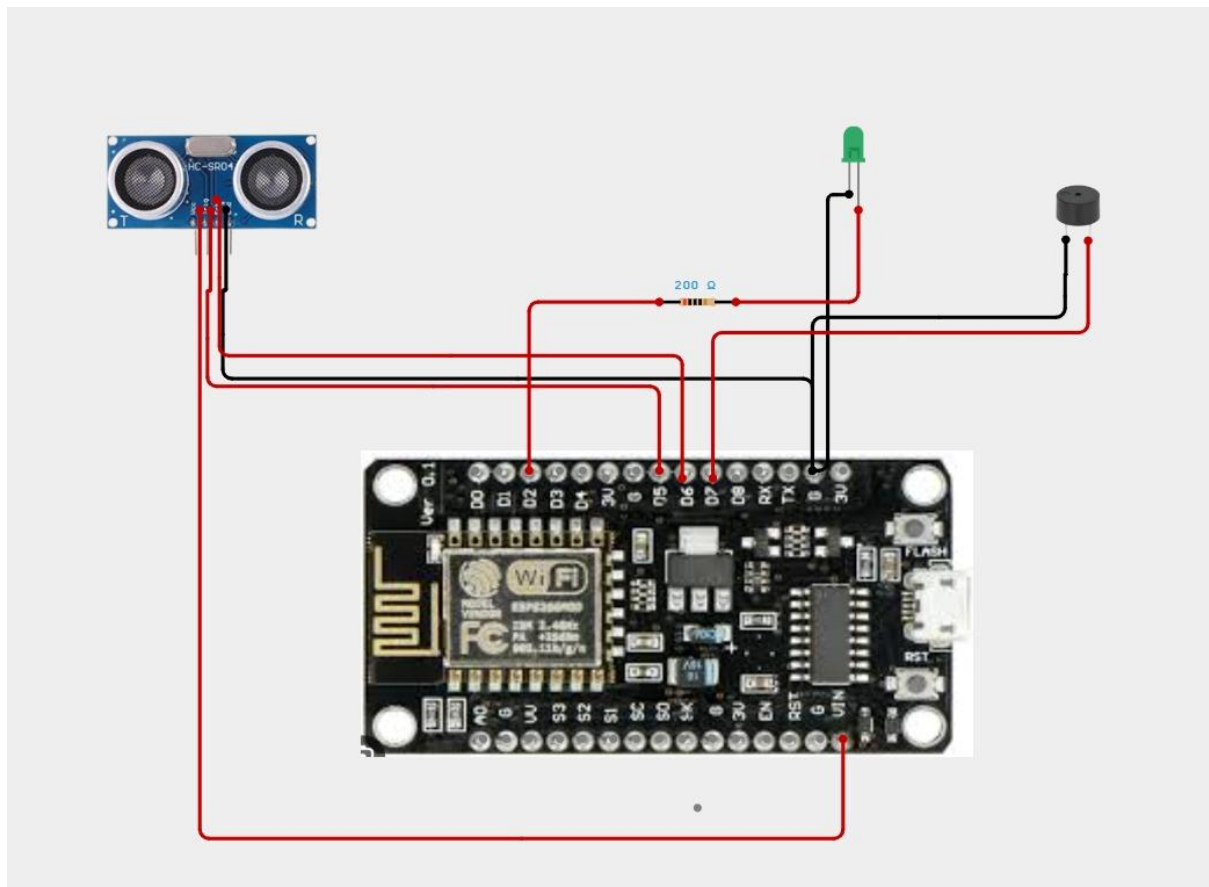
The circuit consists of an ESP8266 NodeMCU, ultrasonic sensor, buzzer, LED, relay module, breadboard, and jumper wires.

The ultrasonic sensor is placed above the water surface. The TRIG pin is connected to GPIO5 (D1), and the ECHO pin is connected to GPIO4 (D2) of the ESP8266 NodeMCU.

The ESP8266 sends a trigger pulse to the ultrasonic sensor. The sensor transmits an ultrasonic wave, which is reflected by the water surface. The ECHO pin provides the time taken for the reflected signal to return. Using this time, the ESP8266 calculates the distance between the sensor and the water surface.

The buzzer is connected to GPIO12 (D6) and provides an audible flood warning. The LED is connected to GPIO13 (D7) through a suitable resistor and provides a visual indication. The relay module is connected to GPIO14 (D5) and can be used to operate an external warning or control device.

When the water level rises and reaches the predefined danger level, the ESP8266 activates the buzzer, LED, and relay.



8. Coding :

```
// WATER LEVEL DETECTION AND FLOOD ALERT SYSTEM

// ESP8266 NodeMCU + HC-SR04 + Buzzer + LED

// + WiFi + ThingSpeak

// =====

#include <ESP8266WiFi.h>

#include <ESP8266HTTPClient.h>

// =====

// WIFI CONNECTION

// =====

const char* WIFI_SSID = "praveen";
const char* WIFI_PASSWORD = "praveen12345";

// =====

// THINGSPEAK API

// =====

const char* THINGSPEAK_API_KEY = "IDONK03HA6Q175BN";
const char* THINGSPEAK_SERVER = "http://api.thingspeak.com/update";

// =====

// HC-SR04 TRIG-> D5 / GPIO14
```

```
// =====
```

```
#define TRIG_PIN D5
```

```
// HC-SR04 ECHO-> D6 / GPIO12
```

```
#define ECHO_PIN D6
```

```
// Buzzer +-> D7 / GPIO13
```

```
#define BUZZER_PIN D7
```

```
// LED Anode-> D2 / GPIO4 through 220 ohm resistor
```

```
#define LED_PIN D2
```

```
// =====
```

```
// TANK / WATER LEVEL SETTINGS
```

```
// =====
```

```
#define TANK_HEIGHT 100
```

```
#define FLOOD_LEVEL 80
```

```
// =====
```

```
// THINGSPEAK UPDATE TIMER
```

```
// =====
```

```
unsigned long lastThingSpeakUpdate = 0;
```



```
const unsigned long THINGSPEAK_INTERVAL = 15000;
```

```
// =====
```

```
// SETUP
```

```
// =====
```

```
void setup()
```

```
{
```

```
  Serial.begin(9600);
```

```
  pinMode(TRIG_PIN, OUTPUT);
```

```
  pinMode(ECHO_PIN, INPUT);
```

```
  pinMode(BUZZER_PIN, OUTPUT);
```

```
  pinMode(LED_PIN, OUTPUT);
```

```
  digitalWrite(TRIG_PIN, LOW);
```

```
  digitalWrite(BUZZER_PIN, LOW);
```

```
  digitalWrite(LED_PIN, LOW);
```

```
// =====
```

```
// WIFI CONNECTION
```

```
// =====
```

```
  Serial.println();
```

```
Serial.println("=====");
```

```
Serial.println(" WATER LEVEL & FLOOD ALERT SYSTEM");
```

```
Serial.println("=====");
```

```
Serial.println("Connecting to WiFi...");
```

```
WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
```

```
while (WiFi.status() != WL_CONNECTED)
```

```
{
```

```
    delay(500);
```

```
    Serial.print(".");
```

```
}
```

```
Serial.println();
```

```
Serial.println("WiFi Connected!");
```

```
Serial.print("IP Address: ");
```

```
Serial.println(WiFi.localIP());
```

```
// =====
```

```
// ORIGINAL SYSTEM INITIALIZATION
```

```
// =====
```

```
Serial.println("System Ready");
```

```
Serial.println("-----");
```

```
}
```

```
// =====
```

```
// LOOP
```

```
// =====
```

```
void loop()
```

```
{
```

```
// =====
```

```
// SEND ULTRASONIC PULSE
```

```
// =====
```

```
digitalWrite(TRIG_PIN, LOW);
```

```
delayMicroseconds(2);
```

```
digitalWrite(TRIG_PIN, HIGH);
```

```
delayMicroseconds(10);
```

```
digitalWrite(TRIG_PIN, LOW);
```

```
// Read echo time
```

```
long duration = pulseIn(ECHO_PIN, HIGH, 30000);
```

```
// Calculate distance
```

```
float distance = duration * 0.0343 / 2;
```

```
// =====
```

```
// CHECK SENSOR READING
```

```
// =====
```

```
if (duration == 0)
```

```
{
```

```
    Serial.println("Ultrasonic Sensor Error!");
```

```
    digitalWrite(BUZZER_PIN, LOW);
```

```
    digitalWrite(LED_PIN, LOW);
```

```
    delay(1000);
```

```
    return;
```

```
}
```

```
// =====
```

```
// CALCULATE WATER LEVEL
```

```
// =====
```

```
float waterLevel = TANK_HEIGHT- distance;
```

```
// Prevent negative water level
```

```
if (waterLevel < 0)
```

```
{
```

```
    waterLevel = 0;
```

```
}
```

```
// Prevent water level exceeding tank height
```

```
if (waterLevel > TANK_HEIGHT)
```

```
{
```

```
    waterLevel = TANK_HEIGHT;
```

```
}
```

```
// =====
```

```
// SERIAL MONITOR
```

```
// =====
```

```
Serial.print("Distance from Sensor: ");
```

```
Serial.print(distance);
```

```
Serial.println(" cm");
```

```
Serial.print("Water Level: ");
```

```
Serial.print(waterLevel);
```

```
Serial.println(" cm");
```

```
// =====
```

```
// FLOOD ALERT
```

```
// =====
```

```
if (waterLevel >= FLOOD_LEVEL)
```

```
{
```

```
    digitalWrite(LED_PIN, HIGH);
```

```
digitalWrite(BUZZER_PIN, HIGH);

Serial.println("!!! FLOOD WARNING !!!");
Serial.println("Water Level: HIGH");
Serial.println("Buzzer: ON");
Serial.println("LED: ON");
}
else
{
digitalWrite(LED_PIN, LOW);
digitalWrite(BUZZER_PIN, LOW);

Serial.println("Water Level: NORMAL");
Serial.println("Buzzer: OFF");
Serial.println("LED: OFF");
}

Serial.println("-----");

// =====
// SEND WATER LEVEL TO THINGSPEAK
// =====

if (millis()- lastThingSpeakUpdate >= THINGSPEAK_INTERVAL)
{
lastThingSpeakUpdate = millis();
```

```
if (WiFi.status() == WL_CONNECTED)
{
    WiFiClient client;
    HTTPClient http;

    String url = String(THINGSPEAK_SERVER);
    url += "?api_key=";
    url += THINGSPEAK_API_KEY;
    url += "&field1=";
    url += String(waterLevel);

    Serial.println("Sending data to ThingSpeak...");

    http.begin(client, url);

    int httpCode = http.GET();

    if (httpCode > 0)
    {
        Serial.print("ThingSpeak Response: ");
        Serial.println(httpCode);

        if (httpCode == 200)
        {
            Serial.println("ThingSpeak Update Successful!");
        }
    }
}
```

```
    }  
    else  
    {  
        Serial.println("ThingSpeak Update Failed!");  
    }  
}  
else  
{  
    Serial.print("ThingSpeak Error: ");  
    Serial.println(http.errorToString(httpCode));  
}  
  
    http.end();  
}  
else  
{  
    Serial.println("WiFi Disconnected!");  
}  
  
    Serial.println("_____");  
}  
  
    delay(1000);  
}
```


CODING OUTPUT:

```
sketch_aug14b | Arduino 1.8.16
File Edit Sketch Tools Help

sketch_aug14b

// =====
// WATER LEVEL DETECTION AND FLOOD ALERT SYSTEM
// ESP8266 NodeMCU + HC-SR04 + Buzzer + LED
// + WiFi + ThingSpeak
// =====

#include <ESP8266WiFi.h>
#include <ESP8266HTTPClient.h>

// =====
// WIFI CONNECTION
// =====

const char* WIFI_SSID = "praveen";
const char* WIFI_PASSWORD = "praveen12345";

// =====
// THINGSPEAK API
// =====

const char* THINGSPEAK_API_KEY = "IDONK03RAGU1758M";
const char* THINGSPEAK_SERVER = "http://api.thingspeak.com/update";

// =====
// HC-SR04 TRIG -> D5 / GPIO14
// =====

#define TRIG_PIN D5

// HC-SR04 ECHO -> D6 / GPIO12
#define ECHO_PIN D6

// Buzzer + -> D7 / GPIO13
#define BUZZER_PIN D7

// LED Anode -> D2 / GPIO4 through 220 ohm resistor
#define LED_PIN D2

// =====
// TANK / WATER LEVEL SETTINGS
// =====

Done uploading.
```

```
COM6
Distance from Sensor: 10.46 cm
Water Level: 89.54 cm
!!! FLOOD WARNING !!!
Water Level: HIGH
Buzzer: ON
LED: ON
-----
Distance from Sensor: 9.67 cm
Water Level: 90.33 cm
!!! FLOOD WARNING !!!
Water Level: HIGH
Buzzer: ON
LED: ON
-----
Sending data to ThingSpeak...

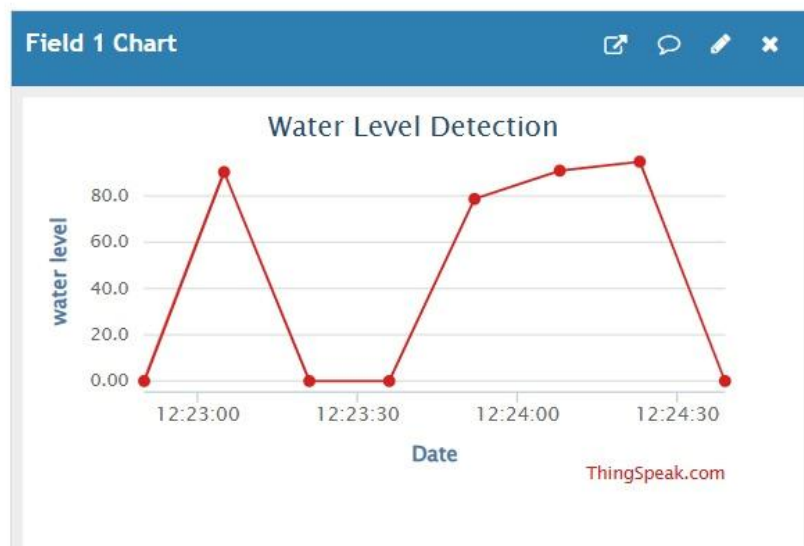
Autoscroll Show timestamp Newline 9600 baud Clear output
```

Channel Stats

Created: 5.minutes.ago

Last entry: less than a minute ago

Entries: 8



9. System Working :

Power ON the System

The ESP8266 NodeMCU is powered ON.

Initialize Components

The PIR sensor, servo motor, buzzer, and LED are initialized.

Measure Distance

- The ESP8266 sends a trigger signal to the ultrasonic sensor.
- The ultrasonic sensor sends an ultrasonic wave towards the water surface.
- The ultrasonic wave is reflected from the water surface.
- The sensor receives the reflected signal.

Calculate Water Distance

The ESP8266 calculates the distance between the ultrasonic sensor and the water surface using the echo time.

Process the Data

The ESP8266 receives and processes the human detection signal.

Determine Water Level

As the water level rises, the distance between the sensor and water surface decreases.

Check Water Level

The measured distance is compared with the predefined danger level.

High Water Level Condition

If the water reaches the danger level:

- Buzzer → ON
- LED → ON
- Relay → ON

- Flood alert → Activated

Normal Water Level Condition

If the water level is within the safe range:

- Buzzer → OFF
- LED → OFF
- Relay → OFF
- System remains in monitoring mode.

Repeat the Process

- The ultrasonic sensor continuously measures the distance, and the ESP8266 continuously monitors the water level.

10. PROTOTYPE IMPLEMENTATION :

The prototype is implemented using an ESP8266 NodeMCU, ultrasonic sensor, relay module, buzzer, LED, breadboard, and jumper wires. The ultrasonic sensor is mounted above a water container to continuously measure the distance between the sensor and the water surface.

Hardware Setup

1. Connect the Ultrasonic Sensor TRIG pin → D1 (GPIO5) of ESP8266.
2. Connect the Ultrasonic Sensor ECHO pin → D2 (GPIO4) of ESP8266.
3. Connect the Relay IN pin → D5 (GPIO14).
4. Connect the Buzzer → D6 (GPIO12).
5. Connect the LED → D7 (GPIO13) through a suitable resistor.
6. Connect the Ultrasonic Sensor VCC → suitable power supply.
7. Connect all GND terminals to a common ground.
8. Connect all required low-voltage components using jumper wires.
9. Upload the program to the ESP8266 using the Arduino IDE.
10. Mount the ultrasonic sensor above the water container.
11. Power ON the prototype and observe the distance readings.
12. Increase the water level gradually.
13. When the water reaches the predefined danger level, observe the activation of the buzzer, LED, and relay.
14. The system continuously monitors the water level.

Prototype Operation

The prototype demonstrates automatic water-level monitoring and flood warning. The ultrasonic sensor provides the distance measurement, while the ESP8266 processes the sensor readings and controls the buzzer, LED, and relay based on the programmed condition.

Prototype Working :

- The ESP8266 NodeMCU is powered ON and initializes all connected components.
- The ultrasonic sensor continuously measures the distance between the sensor and the water surface.
- The sensor sends ultrasonic pulses towards the water surface.
- The reflected echo signal is received by the sensor.
- The ESP8266 calculates the distance using the echo time.
- The calculated distance is used to determine the water level.
- As the water level rises, the distance measured by the ultrasonic sensor decreases.
- The measured distance is compared with the predefined flood danger level.
- If the water reaches the danger level, the buzzer turns ON to alert the user.
- At the same time, the LED provides a visual warning indication.
- The relay connected to D5 (GPIO14) is activated for suitable warning or control purposes.
- If the water level is within the normal limit, the buzzer and LED remain OFF, and the relay stays in its normal state.
- The system waits for a short interval and measures the water level again.
- This process continues automatically and continuously, providing real-time water-level monitoring and flood alerts.

11. Bill of Materials :

S.No.	Component	Specification	Quantity
1	ESP8266	NodeMCU Wi-Fi Development Board	1
2	Ultrasonic Sensor	HC-SR04 or Equivalent	1
3	Relay Module	1-Channel Relay	1
4	Buzzer	3.3V/5V Buzzer	1
5	LED	Standard Indicator LED	1
6	Resistor	Suitable LED Current Limiting Resistor	1
7	Breadboard	Standard Prototype Board	1
8	Jumper Wires	Male-Male / Male-Female	1 Set
9	USB Cable	Micro-USB	1
10	Power Supply	Suitable Regulated Supply	1
11	Water Container	Prototype Water Storage Model	1

12. Results & Performance Analysis :

- The ultrasonic sensor successfully measures the distance between the sensor and the water surface.
- The ESP8266 NodeMCU successfully receives and processes the distance measurement.
- The system provides continuous monitoring of the water level.
- As the water level rises, the measured distance decreases.
- When the water reaches the predefined danger level, the buzzer is activated.
- The LED provides a visual flood warning indication.

- The relay is activated to demonstrate an external warning or control operation.
- When the water level returns to the safe range, the buzzer and LED turn OFF.
- The system provides an automatic and continuous water-level monitoring and flood alert process.

Results :

- The ultrasonic sensor successfully measures the distance to the water surface.
- The ESP8266 NodeMCU successfully receives and processes the sensor data.
- The system continuously monitors the water level.
- The system successfully detects changes in the water level.
- When the water reaches the predefined danger level, the buzzer turns ON.
- The LED provides a visual warning indication.
- The relay activates to demonstrate flood warning or control.
- When the water level returns to the normal range, the buzzer and LED turn OFF.
- The prototype demonstrates the basic operation of a Water Level Detection and Flood Alert System.

Performance Analysis

Parameter	Expected Performance
Water Level Monitoring	Continuous
Distance Measurement	Real-time
Sensor Response	Immediate Reading
High Water Level Alert	Buzzer ON
Visual Warning	LED ON
External Control	Relay ON
Normal Condition	Buzzer OFF and LED OFF
Door Closing Automatic After Delay	Door Closing Automatic After Delay